

# Agentic-SQL Revisited: Autonomy-Based Taxonomy and Empirical Benchmark Analysis for LLM Text-to-SQL

No Author Given

No Institute Given

**Abstract.** Text-to-SQL has rapidly evolved from single-pass semantic parsing to increasingly interactive and autonomous reasoning pipelines driven by large language models (LLMs). However, many studies are still organized mainly by training paradigm, which obscures an equally important question: how inference autonomy changes system behavior across benchmarks of different realism and difficulty. In this paper, we revisit LLM-based Text-to-SQL through an autonomy-based taxonomy that categorizes representative systems into single-turn generation, iterative refinement, and agentic collaboration. Rather than presenting a descriptive overview alone, we design an analysis framework, construct comparative study settings, and conduct a benchmark-oriented empirical study across Spider, BIRD, and Spider 2.0 through the lens of inference autonomy. Our study highlights three recurring patterns. First, gains on classical academic benchmarks do not transfer proportionally to enterprise-scale settings. Second, execution-guided refinement improves robustness but usually increases inference cost and latency. Third, reasoning-internalized generation provides a competitive middle ground between answer-only generation and externally orchestrated multi-stage prompting. To further ground these observations, we perform a focused case-driven analysis of reasoning internalization through RevDecomp-SFT and examine how explicit intermediate reasoning affects accuracy, efficiency, and diagnosability. This analysis shows that internalized decomposition can remain competitive on both Spider and BIRD while exposing interpretable failure stages such as schema linking and logical decomposition errors. Based on these findings, we discuss the trade-offs among autonomy, interpretability, efficiency, and deployment realism, and outline a benchmark agenda for next-generation Text-to-SQL systems.

**Keywords:** Text-to-SQL · Large Language Models · Taxonomy · Benchmark Analysis · Empirical Study

## 1 Introduction

Text-to-SQL studies whether a system can translate a natural language question into an executable SQL query over a target database. Earlier research largely treated this task as semantic parsing, where progress was measured by how

accurately a model mapped linguistic expressions to SQL structures under unseen schemas. With the adoption of large language models (LLMs), the task has moved beyond narrow parsing assumptions: users now issue more conversational, underspecified, and context-dependent requests, while systems are increasingly expected to operate over large, realistic, and evolving databases.

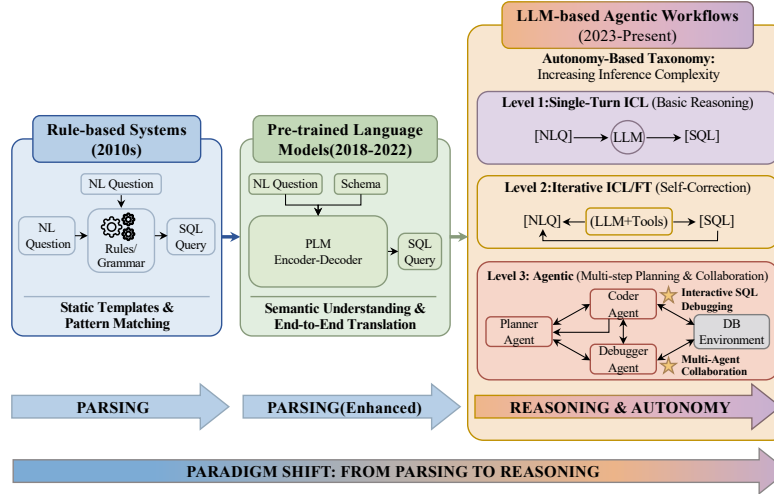


Fig. 1: Evolution of Text-to-SQL from static translation to increasingly autonomous reasoning pipelines. The figure illustrates the shift from single-pass generation to feedback-aware and agentic inference.

This shift has changed what makes modern Text-to-SQL systems succeed or fail. On relatively clean academic benchmarks, strong SQL generation ability can already deliver competitive results. In more realistic settings, however, failures increasingly arise from schema grounding, value disambiguation, incomplete instructions, and the inability to recover from intermediate mistakes. In other words, the central challenge is no longer only whether the model can generate SQL, but whether it can allocate reasoning effort effectively under realistic database constraints.

Current literature is still often organized by training paradigm. While this distinction remains useful, it is no longer sufficient to explain the behavior of recent high-performing systems. Many of the most consequential differences now appear at inference time: some systems still rely on single-pass generation, some revise candidate SQL through iterative feedback, and some decompose the task into modular sub-problems through agentic coordination. These differences materially affect robustness, interpretability, latency, and deployment cost.

Recent studies further suggest that the practical scope of database-oriented language interfaces is already expanding beyond classical Text-to-SQL. On one hand, benchmark work such as Spider 2.0 shows that realistic enterprise workflows require long-horizon reasoning, metadata exploration, dialect adaptation,

and multi-query execution rather than isolated single-shot SQL generation [4]. On the other hand, broader formulations such as Table-Augmented Generation (TAG) argue that many real user requests cannot be reduced to pure SQL translation alone, because they may require a tighter combination of database operations, retrieval, and language-model reasoning [17]. These developments reinforce our view that modern Text-to-SQL systems should be analyzed not only by their training regime, but also by how they allocate inference-time control and reasoning structure.

Motivated by this shift, this paper revisits LLM-based Text-to-SQL through an autonomy-based taxonomy and a benchmark-oriented empirical study. We organize representative systems into three categories: *single-turn generation*, *iterative refinement*, and *agentic collaboration*. We then use this taxonomy as an analytical lens to study how system behavior changes across Spider, BIRD, and Spider 2.0. Rather than treating benchmark scores as isolated outcomes, we design an analysis framework, construct comparative settings, and examine how inference autonomy interacts with schema scale, grounding difficulty, debugging capability, and efficiency constraints.

Our primary contributions are summarized as follows:

- We introduce an autonomy-based taxonomy for LLM Text-to-SQL that organizes representative systems according to their inference behavior rather than only their training setup.
- We design an analysis framework and construct comparative study settings that separate lightweight single-turn generation, externally orchestrated iterative prompting, and reasoning-internalized generation.
- We conduct a benchmark-oriented empirical study across Spider, BIRD, and Spider 2.0, and use this study to characterize how gains on academic benchmarks degrade under larger schemas, more realistic values, and enterprise-style workflows.
- We perform a focused case-driven analysis of reasoning internalization through RevDecomp-SFT, and use this analysis to examine whether explicit intermediate reasoning can offer an effective compromise among accuracy, efficiency, and interpretability.
- We summarize the practical trade-offs among autonomy, interpretability, inference cost, and deployment realism, and derive a benchmark agenda for next-generation Text-to-SQL systems.

The remainder of this paper is organized as follows. Section 2 defines the task setting, benchmark landscape, and evaluation metrics used throughout the paper. Section 3 introduces the proposed autonomy-based taxonomy. Section 4 analyzes representative system families under this taxonomy. Section 5 presents a benchmark-oriented empirical study. Section 6 discusses the implications of these findings for future Text-to-SQL systems, and Section 7 concludes the paper.

## 2 Problem Setting and Benchmarks

This section introduces the task setting, benchmark landscape, and evaluation metrics needed for the later analysis. The purpose is not to provide exhaustive background material, but to establish the minimal context required for interpreting the proposed taxonomy and the benchmark-oriented findings.

### 2.1 Problem Formulation

In Text-to-SQL, the input consists of a natural language question and a database schema, and the output is an executable SQL query that answers the question over the target database. We denote the schema by  $S = \{T, C, R\}$ , where  $T$  is the set of tables,  $C$  is the set of columns, and  $R$  is the set of relations such as foreign-key links. In more realistic settings, the system may additionally rely on external knowledge  $K$  for entity linking, value grounding, or domain-specific disambiguation, as emphasized in BIRD [3].

Under a standard conditional generation view, a model  $\pi$  parameterized by  $\theta$  produces a predicted SQL query  $\hat{Y}$  from instruction  $I$ , question  $Q$ , schema  $S$ , and optional external knowledge  $K$ :

$$\hat{Y} = \pi(I, Q, S, K \mid \theta). \quad (1)$$

This formulation is deliberately generic because the systems studied in this paper differ less in the final objective than in how they reach  $\hat{Y}$ . In ICL-based systems,  $\theta$  remains fixed and performance depends on prompt design and inference structure. In fine-tuned systems, model parameters are adapted to improve SQL generation, schema understanding, or execution-aware reasoning. A standard supervised fine-tuning objective is

$$\mathcal{L}_{SFT} = - \sum_{(P,Y) \in D} \sum_{t=1}^L \log Pr_{\pi}(y_t \mid P, y_{<t}), \quad (2)$$

where  $P$  denotes the input prompt and  $Y$  denotes the gold SQL sequence. Parameter-efficient adaptation methods such as LoRA are particularly relevant in this regime because they make reasoning-oriented specialization practical for open-source backbones [5].

From the perspective of this paper, the key point is that modern Text-to-SQL systems increasingly differ not only in how they are trained, but also in whether generation is performed in a single pass, through iterative revision, or through coordinated multi-step reasoning.

### 2.2 Benchmark Landscape

Progress in Text-to-SQL is closely tied to benchmark evolution. Early datasets such as WikiSQL [1] established the task but mainly involved simple queries over

Table 1: High-level comparison of representative Text-to-SQL benchmarks. The table highlights the shift from structured academic evaluation to more realistic grounding, scalability, and workflow-oriented settings.

| Aspect                         | Spider                    | BIRD                                 | Spider 2.0                         |
|--------------------------------|---------------------------|--------------------------------------|------------------------------------|
| <b>Primary emphasis</b>        | Structured generalization | Realistic grounding                  | Workflow realism & scalability     |
| <b>Task style</b>              | Single-turn NL→SQL        | NL→SQL with stronger value grounding | Multi-step, debugging-aware        |
| <b>Schema scale</b>            | Medium (multi-table)      | Large (wide schemas)                 | Very large / evolving schemas      |
| <b>Value realism</b>           | Moderate                  | High                                 | High                               |
| <b>External knowledge need</b> | Limited                   | Important                            | Often required                     |
| <b>SQL dialect diversity</b>   | Mostly limited            | Mostly limited                       | Frequently multi-dialect           |
| <b>Dominant failure mode</b>   | Compositional SQL errors  | Grounding and scale errors           | Long-horizon and deployment errors |

single tables, making them inadequate for evaluating compositional reasoning or cross-schema generalization.

Spider [2] marked a major step forward by requiring models to generalize across unseen databases with multi-table schemas. It remains a central benchmark because it stresses structured generalization, including joins, nesting, aggregation, and cross-domain transfer. For many years, Spider served as the main reference point for model comparison.

As performance on Spider improved, benchmark design shifted toward more realistic data conditions. BIRD [3] places much stronger emphasis on large schemas, realistic database values, and external-knowledge-dependent grounding. This change is important because it exposes a different class of failures: a system may produce syntactically plausible SQL yet still fail because it misidentifies values, misses relevant schema elements, or incorrectly resolves ambiguous references.

Spider 2.0 extends this trend toward more realistic enterprise-style evaluation [4]. Compared with Spider, it stresses larger and more dynamic schemas, longer reasoning horizons, multi-step workflows, debugging, and dialect-specific execution behavior. As a result, Spider 2.0 is better viewed as testing system robustness under realistic deployment constraints rather than SQL generation alone.

Table 1 summarizes the qualitative shift across these benchmarks. The table is not intended as an exhaustive dataset catalog; instead, it highlights why benchmark interpretation must now consider more than raw SQL correctness. Different benchmarks increasingly probe different system capabilities.

### 2.3 Evaluation Metrics

Evaluation in Text-to-SQL is challenging because multiple SQL queries can be semantically equivalent even when their surface forms differ. For this reason, the field relies on several complementary metrics.

**Exact Match (EM).** EM compares the predicted SQL query  $\hat{Y}$  with the ground-truth query  $Y$  at the string or normalized-structure level. This metric is easy to compute and historically important, but it is also strict: semantically equivalent queries may be penalized if they differ in formatting, aliasing, or alternative but correct SQL constructions [2].

**Execution Accuracy (EX).** EX measures whether the result returned by executing  $\hat{Y}$  matches the result returned by executing the gold SQL. This metric is more aligned with practical use because it captures functional correctness rather than surface similarity. As a result, EX is widely treated as the main metric in modern Text-to-SQL evaluation. However, EX is still imperfect: two logically different queries may occasionally return the same result on a particular database instance.

**Valid Efficiency Score (VES).** VES, introduced in the BIRD setting [3], extends evaluation beyond correctness to execution efficiency. This is especially important in realistic large-scale databases, where a query may be correct but still unusable if it is computationally inefficient.

From the perspective of this paper, these metrics should be interpreted together with system design. A benchmark score does not fully explain how a result was achieved. Two systems may obtain similar EX while differing substantially in latency, reasoning visibility, and robustness under schema or value perturbation. This is one reason why later sections analyze benchmark outcomes jointly with inference autonomy rather than in isolation.

### 3 Autonomy-Based Taxonomy of Text-to-SQL Systems

We next introduce an autonomy-based taxonomy for organizing modern LLM Text-to-SQL systems. The taxonomy is not intended as a descriptive endpoint; rather, it serves as the analytical framework used in the remainder of the paper.

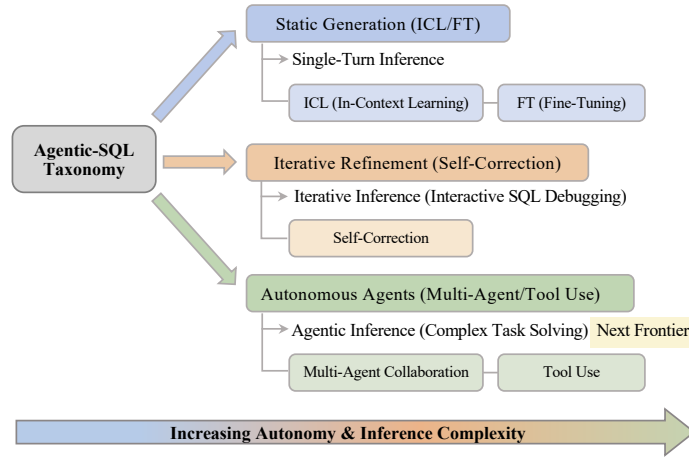


Fig. 2: Autonomy-based taxonomy of Text-to-SQL systems. Representative methods are grouped by the amount of inference-time structure introduced between the input question and the final SQL output.

Rather than organizing LLM Text-to-SQL systems only by training paradigm, we categorize them by *inference autonomy*, that is, how much structured reason-

ing, feedback, and coordination is introduced between the input question and the final SQL output. This perspective is more aligned with current benchmark behavior, because many recent performance differences arise not only from model scale or fine-tuning, but also from how much inference-time control is injected into the pipeline.

We group representative systems into three levels.

**Level 1: Single-Turn Generation.** The model generates SQL in a single forward pass conditioned on the question, schema, and optional demonstrations. Representative systems in this level are efficient and simple to deploy, but are also the most sensitive to schema serialization, example selection, and prompt length. Their main bottleneck is that schema grounding, logical composition, and value disambiguation must all be resolved within one generation step.

**Level 2: Iterative Refinement.** The model is allowed to revise intermediate outputs through feedback. Feedback may come from execution errors, self-verification, or staged subtask decomposition. Compared with Level 1, these systems are more robust to local mistakes because they can repair failed SQL rather than treating first-pass generation as final.

**Level 3: Agentic Collaboration.** The task is decomposed into specialized subtasks such as schema filtering, planning, generation, and review. These systems are most relevant to large and realistic database settings, where single-pass prompting becomes unreliable. Their strength lies in modularity and error recovery; their weakness lies in orchestration complexity, latency, and token cost.

This taxonomy is not introduced as a descriptive endpoint. Instead, it serves as the analytical lens for the benchmark-oriented study in the following sections.

## 4 System Families Under the Taxonomy

Using the proposed taxonomy, we now examine how representative system families differ in inference structure, robustness, and deployment cost. The focus of this section is not to enumerate all methods exhaustively, but to isolate the design patterns that most strongly influence benchmark behavior.

### 4.1 Single-Turn Systems

Single-turn systems remain the most deployment-friendly family because they minimize control overhead and inference latency. In practice, their effectiveness depends heavily on prompt construction. Methods such as DAIL-SQL systematically compare prompt-engineering choices including question representation, example selection, and example organization, then integrate these findings into a token-efficient prompting solution [7]. This line of work is important for our taxonomy because it shows that even when the inference path remains single-turn, system behavior can still vary substantially depending on how prompt structure is designed.

The prompt in this family typically consists of an instruction  $I$ , a schema description  $S$ , and the user question  $Q$ . For zero-shot settings, the prompt is

$$P_0 = I \oplus S \oplus Q \quad (3)$$

and for few-shot settings,  $k$  demonstrations  $E = \{e_1, e_2, \dots, e_k\}$  are prepended:

$$P_k = E \oplus I \oplus S \oplus Q. \quad (4)$$

The model then generates

$$\hat{Y} = \arg \max_Y \prod_{t=1}^L P(y_t \mid y_{<t}, P_k; \theta). \quad (5)$$

Constrained decoding methods such as PICARD further strengthen this regime by explicitly enforcing syntactic validity during autoregressive generation, thereby reducing invalid SQL outputs without introducing iterative correction loops [6]. However, the main limitation of single-turn systems is structural. When the schema is large or the question is ambiguous, the model must simultaneously perform schema linking, logical decomposition, and SQL realization within a single decoding trajectory. This makes such systems increasingly brittle on benchmarks that emphasize realistic values, wide schemas, or incomplete instructions.

## 4.2 Iterative Refinement Systems

Iterative systems partially relax the burden of one-shot generation by allowing the model to revise or re-plan. DIN-SQL is representative in this regard: schema linking, decomposition, and correction are staged rather than collapsed into one pass [8]. This design improves robustness because local failures can be observed and repaired before the final SQL is fixed.

A useful abstraction is to decompose the probability of generating a correct query  $\hat{Y}$  into intermediate steps:

$$P(\hat{Y} \mid P) = P(L \mid P) \cdot P(D \mid L, P) \cdot P(\hat{Y} \mid D, L, P), \quad (6)$$

where  $L$  denotes schema links and  $D$  denotes decomposed sub-queries. By separating schema grounding and logical decomposition from final SQL realization, iterative systems improve both robustness and diagnosability relative to monolithic single-pass decoding.

Subsequent studies reinforce this iterative perspective from different angles. DART-SQL augments prompt-based systems with question rewriting and execution-guided refinement, showing that ambiguity reduction and execution feedback can substantially improve execution accuracy [9]. DTS-SQL further demonstrates that decomposition remains beneficial even for smaller language models, suggesting that the value of iterative structure does not depend solely on backbone scale [10]. More recently, TS-SQL formulates refinement in a test-driven manner, highlighting that self-refinement can be explicitly organized around executable checks rather than only around free-form prompting [11].

From a benchmark-analysis perspective, the importance of iterative systems is not only that they improve execution accuracy, but also that they reveal where single-turn systems fail. In many cases, gains come less from stronger raw generation ability than from making schema grounding and error recovery explicit.



### 4.3 Agentic Systems

Level 3 systems extend staged reasoning into modular coordination. Frameworks such as MAC-SQL and related agentic pipelines treat Text-to-SQL as a coordinated process involving planning, coding, verification, and sometimes tool use or agent evolution [12]. This family is most promising for realistic database environments because it is structurally better suited to long-horizon reasoning and large schemas.

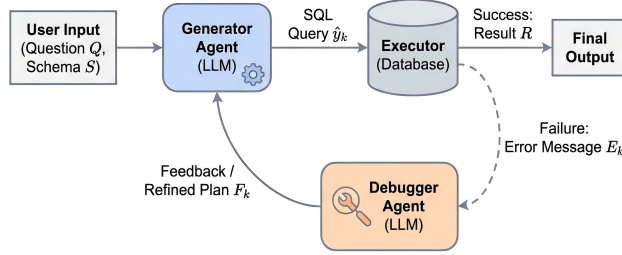


Fig. 3: A representative agentic reasoning loop for Text-to-SQL. The pipeline combines planning, execution, and correction to improve robustness on complex database tasks.

Execution-guided refinement is one of the most direct steps toward inference autonomy. Instead of assuming that the first generated SQL query is correct, the system executes it and uses feedback from the database engine to determine the next action. Let  $Y_t$  be the SQL query generated at iteration  $t$ . The execution function  $E(Y_t, S)$  returns either a result set  $R$  or an error message  $\epsilon$ . If an error occurs, the next query is generated as

$$Y_{t+1} = \pi(I, Q, S, Y_t, \epsilon \mid \theta). \quad (7)$$

Agentic systems can also distribute reasoning burden across specialized roles. A planner may identify relevant tables and join conditions, a coder may translate the plan into SQL syntax, and a reviewer may verify the output. This interaction can be abstracted as

$$P(Y \mid Q, S) \approx P(Y \mid A_{review}, A_{code}, A_{plan}, Q, S). \quad (8)$$

Recent work also shows that the agentic regime is itself heterogeneous. MAC-SQL represents a multi-agent collaborative formulation in which decomposition, schema reduction, and refinement are handled by specialized roles [12]. ExeSQL extends the execution-driven view toward dialect adaptation by using execution-guided bootstrapping to improve cross-dialect robustness [13]. Meanwhile, EllieSQL highlights a complementary systems insight: since high-autonomy pipelines are expensive, routing mechanisms that allocate different

generation strategies to different query difficulties may be necessary for practical deployment [14].

At the same time, agentic systems expose a central deployment trade-off: correctness is improved through additional control structure, but this control structure itself introduces cost. Therefore, benchmark gains from agentic systems should not be interpreted in isolation from latency, token usage, and orchestration burden.

#### 4.4 Reasoning Internalization as an Intermediate Design

Between explicit multi-stage prompting and pure answer-only generation, an increasingly important direction is to internalize intermediate reasoning into the model through supervision. Under this view, the model is trained to follow or expose intermediate structures such as schema linking, decomposition, draft generation, and revision, while keeping the inference path relatively compact.

This design is analytically important because it occupies a middle point in the autonomy spectrum. It preserves some interpretability and diagnostic structure without fully depending on external multi-call orchestration. RevDecomp-SFT provides a useful case study of this regime by supervising a reasoning trajectory that includes schema linking, logic decomposition, draft generation, revision, and final SQL output. For benchmark-oriented analysis, reasoning internalization should therefore be treated as a distinct system design choice rather than merely a fine-tuning detail.

This intermediate regime is also supported by recent reasoning-oriented training studies. STaR-SQL shows that self-taught reasoning can significantly improve Text-to-SQL performance by converting reasoning trajectories into stronger supervision signals [15]. From our perspective, this is important not only because it improves benchmark accuracy, but because it further validates the broader design hypothesis that some benefits often attributed to external agentic prompting may instead come from making intermediate reasoning explicit and learnable.

## 5 Benchmark-Oriented Empirical Study

This section presents our benchmark-oriented empirical study of modern LLM Text-to-SQL systems under the proposed autonomy-based taxonomy. Rather than treating benchmark scores as isolated outcomes, we analyze recurring performance patterns across settings of increasing realism and examine how inference autonomy affects robustness, efficiency, and failure visibility.

### 5.1 Study Design

Our empirical study is organized around three research questions:

- **RQ1:** How does inference autonomy affect robustness as evaluation moves from Spider to BIRD and Spider 2.0?

- **RQ2:** What do benchmark differences reveal about the limits of single-turn generation?
- **RQ3:** Can reasoning internalization serve as an effective middle ground between one-shot decoding and fully agentic orchestration?

We do not claim a fully unified re-implementation of all systems. Instead, we design an analysis framework that synthesizes benchmark evidence, system behavior, and failure visibility under a common autonomy-based view. This allows us to compare design regimes rather than only comparing isolated leaderboard entries.

## 5.2 Comparative Setting Construction

To make the comparison analytically focused, we construct three comparative settings:

- **Setting A: Lightweight single-turn generation**, represented by constrained decoding or structure-enhanced prompt-based systems that aim to maximize correctness within a compact inference path.
- **Setting B: Externally orchestrated iterative prompting**, represented by multi-stage pipelines that separate schema linking, decomposition, correction, and final SQL generation.
- **Setting C: Reasoning-internalized generation**, represented by systems that encode intermediate reasoning into a single supervised generative process rather than relying on repeated external prompting.

This study design slightly changes the comparison lens used in the original source studies, but it does not modify the reported metric values themselves. The purpose is to compare how different reasoning regimes allocate structure, cost, and interpretability.

## 5.3 Observation 1: Benchmark Gains Do Not Transfer Uniformly

A consistent pattern is that gains on Spider do not transfer proportionally to BIRD and Spider 2.0. Spider primarily stresses cross-domain generalization and SQL compositionality. By contrast, BIRD and Spider 2.0 additionally require realistic value grounding, larger schema handling, more robust disambiguation under incomplete instructions, and, in the case of Spider 2.0, longer-horizon workflows and stronger deployment realism.

This shift changes the failure mode distribution. On Spider, models often fail because of compositional SQL errors. On BIRD-like settings, failures increasingly arise from schema grounding mistakes, value mismatch, missing entity resolution, or incorrect assumptions about the data. On Spider 2.0, these problems are further compounded by workflow complexity and dialect sensitivity. As a result, methods that appear strong on structured academic benchmarks may still exhibit fragile behavior in more realistic environments.

Table 2: Representative studies under the constructed comparative settings. The table summarizes what each setting varies and what empirical insight it contributes to the proposed analysis framework.

| Setting                  | Representative studies             | Main variable                                            | Main insight                                                                                                              |
|--------------------------|------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Setting A                | DAIL-SQL, PICARD                   | Prompt structure, constrained decoding                   | Even without iterative control, prompt organization and syntax constraints materially affect validity and accuracy.       |
| Setting B                | DIN-SQL, DART-SQL, DTS-SQL, TS-SQL | Decomposition, rewriting, execution-guided repair        | Many gains come from exposing schema grounding and correction stages rather than from stronger backbone generation alone. |
| Setting C                | STaR-SQL, RevDecomp-SFT            | Reasoning supervision, internalized decomposition        | Part of the benefit of multi-step reasoning can be retained without fully externalized orchestration.                     |
| High-autonomy deployment | MAC-SQL, ExeSQL, EllieSQL          | Multi-agent coordination, dialect transfer, routing cost | Better robustness is possible, but deployment quality depends on controlling orchestration overhead and cost.             |

#### 5.4 Observation 2: Stronger Autonomy Improves Robustness but Increases Cost

Across system families, stronger inference autonomy generally improves robustness. Iterative refinement can repair local execution errors; agentic coordination can decompose difficult problems into smaller sub-decisions; reasoning-aware pipelines can expose intermediate structures that simplify debugging. These advantages become more important as schema scale and task realism increase.

However, these gains are not free. Additional reasoning steps require more model calls, more prompt tokens, and more control logic. Therefore, the relevant benchmark question is not simply whether a system is more accurate, but whether the added robustness justifies the added computational and engineering overhead.

Table 3 summarizes this trade-off. The table is not intended as a leaderboard substitute; instead, it captures the recurring structural differences that explain why benchmark behavior diverges across system families.

#### 5.5 Additional Studies Under the Comparative Settings

Beyond the focused RevDecomp-SFT case, our comparative settings are further supported by several complementary studies. In the lightweight single-turn regime, DAIL-SQL shows that prompt organization and token efficiency alone can produce substantial gains, while PICARD demonstrates that constrained decoding can improve validity without introducing iterative control [7, 6]. In the iterative regime, DART-SQL, DTS-SQL, and TS-SQL collectively

Table 3: Qualitative comparison of system families under the autonomy-based taxonomy. The table summarizes recurring trade-offs in robustness, interpretability, and deployment cost rather than serving as a leaderboard comparison.

| Dimension              | Single-turn    | Iterative refinement | Agentic collaboration  |
|------------------------|----------------|----------------------|------------------------|
| Inference cost         | Low            | Medium               | High                   |
| Error recovery         | Weak           | Moderate             | Strong                 |
| Interpretability       | Limited        | Moderate             | Strong                 |
| Schema-scale tolerance | Limited        | Moderate             | Strong                 |
| Deployment simplicity  | Strong         | Moderate             | Weak                   |
| Typical failure source | Grounding miss | Incomplete repair    | Orchestration overhead |

show that decomposition, rewriting, and executable self-refinement remain effective across different model scales and refinement mechanisms [9, 10, 11]. In the high-autonomy regime, MAC-SQL and ExeSQL illustrate two distinct paths: one emphasizes agent specialization and modular coordination, while the other emphasizes execution-grounded bootstrapping for dialect transfer [12, 13]. EllieSQL adds a systems perspective by showing that routing should be treated as part of the design space rather than as a post-hoc optimization [14].

Taken together, these studies strengthen two conclusions of our empirical framework. First, the main distinction among modern Text-to-SQL systems is increasingly not only how they are trained, but how much reasoning structure they externalize during inference. Second, improvements in robustness, interpretability, and transfer often come from explicit control over intermediate reasoning stages, but the practical cost of such control differs substantially across system families.

## 5.6 Focused Empirical Study of Reasoning Internalization

To complement the taxonomy-level analysis with a concrete empirical anchor, we perform a focused empirical study of reasoning internalization through RevDecomp-SFT. Our goal is to examine whether internalized decomposition can serve as an effective middle ground between answer-only decoding and externally orchestrated multi-stage prompting.

Specifically, we analyze its benchmark behavior, training configuration, and error patterns on Spider and BIRD, and use this case to study how explicit intermediate reasoning affects robustness, efficiency, and diagnosability. Under the reported training configuration, the student model is fine-tuned with a LoRA-based setup, using an 8B backbone, low-rank adaptation, and a unified autoregressive generation objective that supervises schema linking, logic decomposition, draft SQL generation, revision, and final SQL output within a single inference trajectory [20]. This design is especially relevant to our study because it operationalizes an intermediate regime between compact one-shot generation and fully externalized iterative orchestration.

Table 4: Focused empirical study of reasoning internalization under our comparative setting. The table reorganizes representative benchmark results to compare reasoning-internalized generation against lightweight single-turn and iterative prompting systems in terms of accuracy and efficiency.

| Method                     | Spider EM | Spider EX | BIRD EX | BIRD VES |
|----------------------------|-----------|-----------|---------|----------|
| T5-3B+PICARD               | 71.9      | 75.1      | –       | –        |
| DIN-SQL+GPT-4              | 60.0      | 85.3      | 55.90   | 59.44    |
| RevDecomp-SFT (Qwen2.5-8B) | 75.4      | 86.4      | 57.20   | 61.50    |

Table 4 supports three observations. First, reasoning internalization is competitive even against strong external prompting pipelines. On Spider, RevDecomp-SFT reaches 86.4 EX, exceeding DIN-SQL+GPT-4 by 1.1 points. Second, the gain is not limited to execution correctness: the 75.4 EM also exceeds the T5-3B+PICARD baseline by 3.5 points, suggesting that internalized reasoning improves structural fidelity in addition to functional correctness. Third, the same design remains competitive on BIRD, where RevDecomp-SFT reaches 57.2 EX and 61.5 VES, exceeding DIN-SQL+GPT-4 by 1.3 EX and 2.06 VES, respectively. These patterns support our claim that reasoning internalization is not merely a fine-tuning detail, but a meaningful design point in the autonomy–efficiency trade-off.

The case-driven analysis also sharpens three practical insights. First, explicit intermediate reasoning is valuable not only because it improves accuracy, but because it localizes failure to identifiable stages such as schema linking and logic decomposition. Second, the comparison suggests that some benefits often attributed to multi-call prompting may in fact come from decomposition itself rather than from external orchestration. Third, when a reasoning trace is internalized into the model, interpretability and deployment efficiency need not be treated as mutually exclusive goals.

A further insight comes from error patterns. When intermediate reasoning is explicitly exposed, remaining errors become attributable to specific stages rather than remaining hidden inside a single opaque output. In difficult BIRD cases, the dominant residual failures are no longer simple syntactic breakdowns, but ambiguous schema linking, temporal filtering mistakes, and logically incorrect condition mapping [20]. This observation matters because it changes what should be improved next: not just stronger SQL generation, but stronger grounding and more reliable reasoning supervision.

## 5.7 Study Discussion

Taken together, Spider, BIRD, and Spider 2.0 are no longer testing exactly the same capability. Spider still primarily measures structured SQL generalization. BIRD more strongly measures realistic grounding. Spider 2.0 additionally tests

long-horizon reasoning, dialect adaptation, debugging, and robustness under incomplete context.

This means benchmark interpretation must become more careful. A single aggregate score is no longer sufficient to characterize system quality. Instead, benchmark results should be read together with the system’s reasoning structure, cost profile, and failure mode visibility.

At the same time, our study has a clear boundary. We conduct a benchmark-oriented empirical study and a case-driven analysis under a unified analytical lens, but we do not claim a fully controlled rerun of every system in a single infrastructure. Accordingly, the contribution of this section is to strengthen empirical interpretation and design comparison, not to replace unified reproduction studies. A natural next step would be to re-run representative single-turn, iterative, and reasoning-internalized systems under a strictly standardized protocol.

## 6 Implications and Open Problems

The benchmark patterns above suggest several implications for the design and evaluation of next-generation Text-to-SQL systems. We summarize the most consequential ones in this section.

### 6.1 Implication 1: Benchmarking Must Move Beyond Final Accuracy

If systems reach similar execution accuracy through very different inference strategies, then final accuracy alone becomes an incomplete indicator of quality. Future evaluations should jointly consider correctness, efficiency, and reasoning visibility. A system that achieves its performance only through repeated self-correction or heavy orchestration may be unsuitable for real-time deployment, even if its benchmark score is high.

### 6.2 Implication 2: Adaptive Routing Is Likely Necessary

The benchmark patterns above suggest that no single reasoning regime is optimal for all queries. Easy questions may not need refinement or agentic decomposition, while difficult queries may fail without them. This makes adaptive routing a natural next step: the system should estimate question difficulty and schema complexity, then decide whether to use single-turn generation, iterative repair, or a deeper agentic pipeline. Recent cost-aware routing results such as EllieSQL support this perspective by showing that routing itself should be treated as a first-class design choice [14].

### 6.3 Implication 3: Sequential and Interactive Evaluation Should Be Taken More Seriously

Most mainstream Text-to-SQL benchmarks still emphasize single-turn question answering, even though real usage often involves follow-up clarification, con-

textual dependence, and incremental query refinement. The emergence of sequential benchmarks such as EHR-SeqSQL indicates that interactivity and contextual carry-over are not edge cases, but realistic evaluation dimensions that may favor multi-turn or refinement-aware systems over purely static single-shot pipelines [16].

#### **6.4 Implication 4: Privacy and Scalability Remain Structural Bottlenecks**

Enterprise adoption depends not only on accuracy, but also on whether the system can operate under privacy constraints and very large evolving schemas. Sending full schemas or example rows to external LLM providers may be unacceptable in production. Likewise, many real databases exceed practical context-window limits. Retrieval-augmented schema selection and metadata exploration are therefore not optional extensions, but likely prerequisites for deployment at scale [18, 17].

#### **6.5 Implication 5: Cross-Dialect Generalization Is Still Underdeveloped**

Most research still centers on relatively standardized SQL settings, whereas real systems must operate across dialects such as BigQuery, Snowflake, and ClickHouse. This gap is not just syntactic. It affects window functions, date-time operations, optimization strategies, and debugging behavior. Dialect-aware work such as ExeSQL suggests that cross-dialect robustness will remain a core challenge if Text-to-SQL is to move from benchmark success to practical utility [13].

#### **6.6 Implication 6: Efficiency Is Not a Secondary Concern**

A recurring theme across recent studies is that deployment efficiency should be treated as a primary design constraint rather than an afterthought. Cost-aware routing work such as EllieSQL, together with earlier studies on Text-to-SQL model compression, suggests that real-world deployment quality depends not only on peak benchmark accuracy, but also on whether the reasoning strategy can be delivered under realistic latency and resource budgets [14, 19].

## **7 Conclusion**

This paper revisits LLM-based Text-to-SQL through an autonomy-based taxonomy and a benchmark-oriented empirical study. By organizing representative systems into single-turn generation, iterative refinement, and agentic collaboration, we show that recent progress is increasingly determined by inference behavior rather than by training paradigm alone.

Our study reveals three main conclusions. First, improvements on structured academic benchmarks do not transfer uniformly to more realistic database



settings. Second, stronger inference autonomy generally improves robustness, but introduces non-trivial cost in latency, tokens, and orchestration complexity. Third, reasoning internalization emerges as a meaningful intermediate design point between answer-only decoding and fully externalized agentic pipelines.

Overall, the central lesson is that future Text-to-SQL research should not be evaluated only by final benchmark scores. More informative progress will come from jointly analyzing accuracy, efficiency, interpretability, and deployment realism. We hope the taxonomy, comparative settings, and benchmark-oriented perspective in this paper can help support more rigorous evaluation and more practical system design for next-generation Text-to-SQL.

## Bibliography

- [1] Hwang, W., Yim, J., Park, S., Seo, M.: A comprehensive exploration on WikiSQL with table-aware word contextualization. arXiv preprint arXiv:1902.01069 (2019)
- [2] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., Radev, D.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of EMNLP, pp. 3911–3921 (2018)
- [3] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, R., Geng, R., Huo, N., Zhou, C., Ma, G., Wang, K., Cao, H., Li, J., Shen, F., Zhao, P., Huang, C., Cheng, Y., Zhang, R., Yin, B., Li, C., Zhao, W., Lin, C., Liu, Q., Huang, F., Zhou, J., Qian, L.: Can LLM already serve as a database interface? A big benchmark for large-scale database grounded text-to-SQLs. In: Advances in Neural Information Processing Systems (2023)
- [4] Lei, F., Chen, J., Ye, Y., Cao, R., Shin, D., Su, H., Suo, Z., Gao, H., Hu, W., Yin, P., Zhong, V., Xiong, C., Sun, R., Liu, Q., Wang, S. I., Yu, T.: Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. arXiv preprint arXiv:2411.07763 (2024)
- [5] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: LoRA: Low-rank adaptation of large language models. In: International Conference on Learning Representations (2022)
- [6] Scholak, T., Schucher, N., Bahdanau, D.: PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In: Proceedings of EMNLP, pp. 9895–9901 (2021)
- [7] Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., Zhou, J.: Text-to-SQL empowered by large language models: A benchmark evaluation. arXiv preprint arXiv:2308.15363 (2023)
- [8] Pourreza, M., Rafiei, D.: DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In: Advances in Neural Information Processing Systems (2023)
- [9] Mao, W., Wang, R., Guo, J., Zeng, J., Gao, C., Han, P., Liu, C.: Enhancing text-to-SQL parsing through question rewriting and execution-guided refinement. In: Findings of the Association for Computational Linguistics: ACL 2024, pp. 2009–2024 (2024)
- [10] Pourreza, M., Rafiei, D.: DTS-SQL: Decomposed text-to-SQL with small large language models. In: Findings of the Association for Computational Linguistics: EMNLP 2024, pp. 8212–8220 (2024)
- [11] Xu, W., Zhu, H., Yan, L., Liu, C., Han, P., Duan, S., Pan, J. Z.: TS-SQL: Test-driven self-refinement for text-to-SQL. In: Findings of the Association for Computational Linguistics: EMNLP 2025 (2025)
- [12] Wang, B., Ren, C., Yang, J., Liang, X., Bai, J., Zhang, Q.-W., Yan, Z., Li, Z.: MAC-SQL: A multi-agent collaborative framework for text-to-SQL. arXiv preprint arXiv:2312.11242 (2023)

- [13] Zhang, J., Yang, H., Miao, K., Zhang, R., Pi, R., Gao, J., Zhou, X.: ExeSQL: Self-taught text-to-SQL models with execution-driven bootstrapping for SQL dialects. In: Findings of the Association for Computational Linguistics: EMNLP 2025, pp. 24305–24326 (2025)
- [14] Zhu, Y., Jiang, R., Li, B., Tang, N., Luo, Y.: EllieSQL: Cost-efficient text-to-SQL with complexity-aware routing. OpenReview (2025)
- [15] He, M., Shen, Y., Zhang, W., Peng, Q., Wang, J., Lu, W.: STaR-SQL: Self-taught reasoner for text-to-SQL. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (2025)
- [16] Ryu, J., Cho, S., Lee, G., Choi, E.: EHR-SeqSQL: A sequential text-to-SQL dataset for interactively exploring electronic health records. In: Findings of the Association for Computational Linguistics: ACL 2024, pp. 16388–16407 (2024)
- [17] Biswal, A., Patel, L., Jha, S., Kamsetty, A., Liu, S., Gonzalez, J. E., Guestrin, C., Zaharia, M.: Text2SQL is not enough: Unifying AI and databases with TAG. arXiv preprint arXiv:2408.14717 (2024)
- [18] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems 33, pp. 9459–9474 (2020)
- [19] Sun, S., Gao, Y., Zhang, Y., Su, J., Chen, B., Lin, Y., Sun, S.: An exploratory study on model compression for text-to-SQL. In: Findings of the Association for Computational Linguistics: ACL 2023, pp. 11647–11654 (2023)
- [20] Anonymous: RevDecomp-SFT: Reverse-distilled decomposition for reasoning-internalized text-to-SQL. Manuscript / source paper used for focused case-driven analysis (2026)  
*Replace this placeholder entry with the exact bibliographic metadata of your source paper.*